

A Physics-Informed Neural Network for Compressible Rayleigh-Taylor Mixing

Michael D. Walker,* S. Trevor Fush, & Michael E. Mueller
Princeton University, USA

Abstract

This study investigates the capability of Physics-Informed Neural Networks (PINNs) to reproduce the two-dimensional, compressible Rayleigh-Taylor instability. A deep feed-forward neural network is trained for the compressible Navier-Stokes equations without labeled data, relying solely on a composite loss function derived from the governing partial differential equations (PDEs) and associated initial and boundary conditions. The PINN results are benchmarked against *Miranda*, a high-order finite-difference implicit Large Eddy Simulation (iLES) solver. While the PINN successfully captures the linear growth phase and large-scale mean flow features, it fails to resolve the fine-scale turbulent structures characteristic of high Reynolds number mixing. This limitation is attributed to the “spectral bias” of fully connected neural networks, which preferentially learn low-frequency components. Consequently, the PINN behaves analogously to a flow with artificially high viscosity.

1 Introduction

Buoyancy-driven Rayleigh-Taylor Instability (RTI) occurs when a less dense fluid is continuously accelerated towards a more dense fluid, a result of the conversion of gravitational potential energy into kinetic energy. This mixing is fundamental to processes ranging from Type 1a supernovae and geological flows to inertial confinement fusion (Zhou (2017)). Accurately predicting the mixing layer growth and transition to turbulence is a classical challenge in computational fluid dynamics.

Traditional grid-based methods, such as high-order finite-difference schemes, are computationally expensive for high-resolution turbulent flows. Physics-Informed Neural Networks (PINNs, Raissi et al. (2019)) offer a mesh-free paradigm by encoding the governing equations directly into the loss function of a neural network. However, resolving multi-scale phenomena like turbulence remains difficult. Neural networks exhibit “spectral bias” (Rahaman et al. (2019)), converging slowly on high-frequency solution components. This work assesses whether a standard PINN can capture compressible RTI physics by comparing the results against *Miranda* (Cook (2007); Olson (2023)), a high-order finite-difference iLES solver. RTI provides a compelling test problem to investigate whether the PINNs can at least capture the early-stage linear growth and subsequent mean flow characteristics, such as the mixing layer growth rate and mixture fraction profiles, even if they fail to resolve later-stage turbulence.

2 Methodology

2.1 GOVERNING EQUATIONS. The analysis considers the 2-D compressible multi-component Navier-Stokes equations for density ρ , velocity $\mathbf{u} = (u, v)$, pressure p , and species mass fractions Y_k . Two miscible species ($N_s = 2$) are considered: a heavy fluid Y_h and a light fluid Y_ℓ , with calorically perfect monoatomic gas properties (i.e., $\gamma = 5/3$). The conservation laws for an inviscid, adiabatic flow are:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad ; \quad \frac{\partial \rho Y_k}{\partial t} + \nabla \cdot (\rho Y_k \mathbf{u}) = 0, \quad k = 1, \dots, N_s$$

*Corresponding author: michael.walker@princeton.edu

$$\begin{aligned}\frac{\partial \rho \mathbf{u}}{\partial t} + \nabla \cdot (\rho \mathbf{u} \mathbf{u} + p \delta) &= \rho \mathbf{g} \\ \frac{\partial E_t}{\partial t} + \nabla \cdot [(E_t + p) \mathbf{u}] &= \rho \mathbf{g} \cdot \mathbf{u}\end{aligned}$$

where $E_t = p/(\gamma - 1) + \rho \mathbf{u} \mathbf{u}/2$ is total energy, δ is the Kronecker delta, and $\mathbf{g}$ is gravitational acceleration. Small artificial viscosities and diffusivities are used for numerical stability in the iLES comparison.

2.2 PINN FRAMEWORK. The solution is approximated by a fully connected feed-forward neural network $\mathcal{N}(t, x, y; w_i)$, which outputs the primitive variables $\phi \in (\rho, u, v, p, Y_h)$. Positivity is enforced by predicting latent variables $\ln \rho, \ln p$, and using a sigmoid activation for Y_h . The network is trained by minimizing the squared error loss $\mathcal{L}$ comprising the PDE residuals, initial conditions (IC), and boundary conditions (BC):

$$\mathcal{L} = \omega_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \omega_{\text{IC}} \mathcal{L}_{\text{IC}} + \omega_{\text{BC}} \mathcal{L}_{\text{BC}} . \quad (1)$$

The PDE residuals f are computed via automatic differentiation. The total PDE loss is the sum of squared residuals over N collocation (training) points : $\mathcal{L}_{\text{PDE}} = \sum_N (f_{\text{mass}}^2 + f_{\text{species}}^2 + f_{\text{mom},x}^2 + f_{\text{mom},y}^2 + f_{\text{energy}}^2)$.

2.3 IMPLEMENTATION DETAILS. The instability is initialized in a dimensionless domain $L_x = 2.8\pi, L_y = 2\pi$ with gravity $g_x = -0.01$. $\mathcal{L}_{\text{BC}}$ is computed as the sum of squared primitive variables' difference from the boundary conditions (no-slip/no-penetration in x , periodic in y). The initial interface is perturbed by a sinusoid $\eta(y) = 0.5 \sin(4y) + L_x/2$ and diffused with a hyperbolic tangent profile. The Atwood number is $A_t \equiv (\rho_h - \rho_\ell)/(\rho_h + \rho_\ell) = 0.5$. $\mathcal{L}_{\text{IC}}$ is computed as the sum of squared primitive variables' departure from the initially quiescent state $\mathbf{u}_0 = 0$ and analytic pressure and density profiles at hydrostatic equilibrium.

- **Architecture:** 8 hidden layers, 128 neurons each, totaling 116,741 trainable parameters at a total size of 0.48 MB. The hyperbolic tangent (tanh) activation function is infinitely differentiable, precise for computing the derivatives required by PDE residuals. The architecture was chosen as a standard baseline derived from Raissi et al. (2019). It provides sufficient depth to model the non-linearities of the governing equations while remaining computationally manageable. Optimization plateaus indicate that network capacity is not limiting. The network was implemented¹ in PyTorch (Bafghi and Raissi (2023)).
- **Training:** Optimization used Adam with an initial learning rate of 1×10^{-4} . Training required approximately 100,000 epochs, as shown in Figure 1 and 6-8 hours of wall-time on one Intel Sapphire Rapids CPU node with 4 NVIDIA A100 GPUs ($\approx 10 \times$ GPU acceleration). The cost is dominated by the automatic differentiation graph traversal for derivatives. Collocation (training) points were generated with Latin Hypercube sampling (interior: $N_{\text{PDE}} = 25,000$, initial condition: $N_{\text{IC}} = 10,000$, wall and periodic boundary: $N_{\text{BC}} = 7500$) and resampled every 1,000 epochs. Loss weights were dynamically adjusted to prioritize the PDE loss ($\omega_{\text{PDE}}, \omega_{\text{IC}}, \omega_{\text{BC}} = 10.0$) after 10,000 epochs of initial boundary fitting.
- **Benchmark:** Verification data was generated on a 256×256 grid using *Miranda*, a 10th-order compact finite-difference iLES solver with localized artificial diffusivity for stability (Cook (2007); Olson (2023)).

3 Results

3.1 FLOW FIELD COMPARISON. Figure 2 compares the evolution of the density field and flow variables averaged in y (periodic direction). Shown are profiles every $t = 10$ [sec] from *Miranda* and the final value captured by the PINN over $t \in [0, 100]$ seconds. The mixing ratio ($\theta \in [0, 1]$, $\theta = 0$ fully segregated, $\theta = 1$ fully mixed) is calculated as $\theta(t) = 1 - 4 \int \overline{Y_h Y_\ell} dx / h(t)$. *Miranda* resolves fine-scale turbulent structures and secondary vortices. In contrast, the PINN captures the global symmetry and stratification, but produces

¹The code library for this implementation is available at the repository: <https://github.com/mw6136/RT-PINNs>.

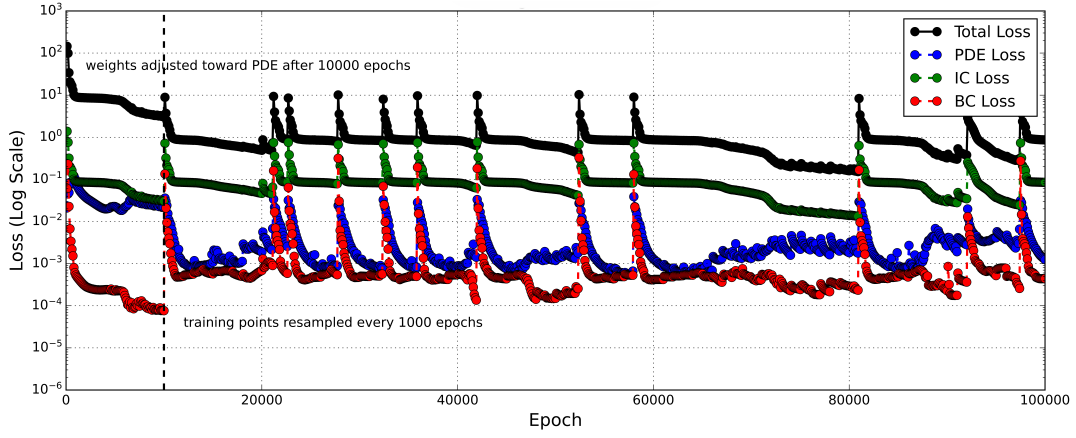


Figure 1: Evolution of the constituent loss terms (PDE, IC, BC) and Total Loss over 100,000 epochs. Note the rapid convergence of BCs contrasted with the slow, plateauing decay of the PDE loss, indicative of the optimization difficulty in the domain interior. Collocation points were resampled every 1,000 epochs for generalization, causing sharp increases in error in some instances.

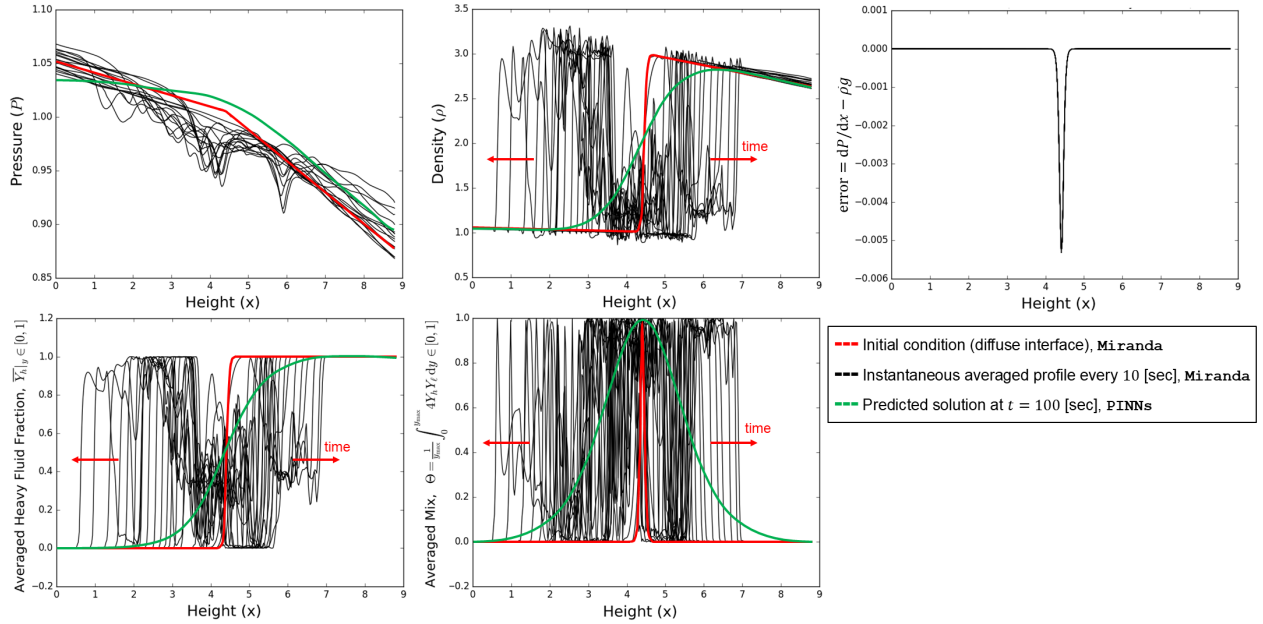


Figure 2: Instantaneous profiles of mixture and flow variables averaged over the domain in the periodic direction. Black lines show the broadening of the mixing layer in time. With the PINN's prediction (green) at late time, the interface remains coherent but highly diffusive.

a highly diffused solution. The vorticity field analysis (not shown) confirms that the PINN captures the shear layers at the bubble-spike interface but lacks small-scale vortices. This supports the spectral bias hypothesis; the network acts as a low-pass filter, effectively simulating a flow with high artificial viscosity.

3.2 GROWTH RATE. Despite the lack of fine-scale resolution, the PINN accurately predicts macroscopic parameters. The mixing layer width $h(t) = 4 \int \bar{Y}_h (1 - \bar{Y}_h) dx$ is compared in Figure 3. The PINN prediction follows the theoretical quadratic growth $h \approx \alpha A_t g t^2$ (Cook et al. (2004)), indicating that the PINN captures the fundamental energy conversion driving the instability. However, the iLES results show a much more clear transition between growth regimes (exponential, drag-dominant, and turbulent, self-similar growth).

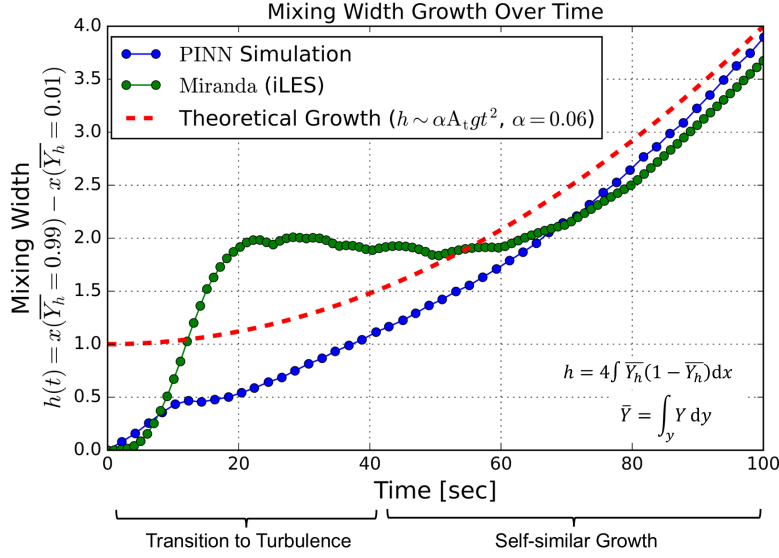


Figure 3: Mixing width $h(t)$ predicted by the PINN compared to iLES and theoretical growth ($h \sim t^2$). $\alpha = 0.06$ is a typical value for the “self-similarity” parameter in 2-D RTI.

4 Conclusion

This work demonstrates that, while standard PINNs can represent the compressible Navier-Stokes equations and capture the macro-scale dynamics of Rayleigh-Taylor instability, the method struggles to resolve the high-frequency spectrum required for turbulence. The resulting solution resembles an ensemble-averaged flow. Future work to make PINNs viable for high-fidelity simulations of turbulence must incorporate spectral bias mitigation strategies, such as Fourier feature mapping or multi-scale network architectures.

Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Department of Energy Computational Science Graduate Fellowship under Award Number DE-SC0024386. The work presented was substantially performed using the Research Computing resources at Princeton University.

References

- R. A. Bafghi and M. Raissi. PINNs-Torch: Enhancing speed and usability of physics-informed neural networks with PyTorch. In *The Symbiosis of Deep Learning and Differential Equations III*, 2023.
- A. W. Cook. Artificial fluid properties for large-eddy simulation of compressible turbulent mixing. *Physics of Fluids*, 19(5):055103, 2007. ISSN 1070-6631. doi: 10.1063/1.2728937.
- A. W. Cook, W. Cabot, and P. L. Miller. The mixing transition in Rayleigh-Taylor instability. *Journal of Fluid Mechanics*, 511:333–362, 2004. doi: 10.1017/S0022112004009681.
- B. J. Olson. PYRANDA: A Python driven, Fortran powered Finite Difference solver for arbitrary hyperbolic PDE systems and mini-app for the LLNL Miranda code, 2023. URL <https://github.com/LLNL/pyranda>.
- N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, and A. Courville. On the spectral bias of neural networks. *Proceedings of the 36th International Conference on Machine Learning*, 97:5301–5310, 2019.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *Journal of Computational Physics*, 378:686–707, 2019. ISSN 0021-9991. doi: 10.1016/j.jcp.2018.10.045.
- Y. Zhou. Rayleigh-Taylor and Richtmyer-Meshkov instability induced flow, turbulence, and mixing. I. *Physics Reports*, 720–722:1–136, 2017. ISSN 0370-1573. doi: 10.1016/j.physrep.2017.07.005.